■ Design WhatsApp / Messenger

System Design Interview | Real-Time Communication

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	1-1 messaging; Group chats (up to 1000 members); Online presence; Message delivery receipts (sent/delivered/read); Media sharing
Non-Functional	2B users; 100B messages/day; <100ms message delivery; End-to-end encryption; 99.99% availability
Scale	~1M msg/sec peak; Each message ~100 bytes text = ~10 TB/day; Media stored separately in object storage

■ High-Level Architecture

• Clients connect via persistent WebSocket connections to Chat Servers; long-lived connections enable real-time push

• Connection Registry (Redis) maps user_id → server_id; when message arrives, lookup recipient's server then push

• Message Queue (Kafka) for async fanout, retry logic, and guaranteed delivery; Chat servers consume from queue

• Zookeeper/Service Discovery for chat server health; failed server → clients reconnect to new server

■ Message Flow Deep Dive

• **Send Flow:** Client → WebSocket → Chat Server → persist to DB (Cassandra) → publish to Kafka topic → consumed by recipient's server → push to recipient via WebSocket

• **Offline User:** Message stored in DB with status=PENDING; when user comes online, server fetches undelivered messages; push notifications via APNS/FCM for mobile wake-up

• **Message ID:** Use Snowflake IDs (timestamp+server+seq) for globally unique, time-sorted message IDs; enables efficient pagination

• **Delivery Receipts:** Sent (stored in DB), Delivered (recipient's device ACKed), Read (recipient opened chat); each status update flows back to sender

■ Data Model (Cassandra)

• messages table: partition_key=(chat_id), clustering_key=(message_id DESC) → enables efficient range scan by conversation

• chat_members table: partition_key=(chat_id) → quick member lookup for group message fanout

• user_connections table in Redis: user_id → {server_id, last_seen, status} with TTL for online presence

• Media: store in S3/CDN; message stores only media_url + thumbnail; client downloads on demand

■■ Trade-offs

WebSocket (Chosen)	Long Polling / SSE
✓ Full-duplex real-time	✓ Simpler server-side
✓ Lower overhead per message	✓ Works through all proxies
✓ Ideal for chat	✓ One-direction (SSE)
✓ Complex connection management	✓ Higher latency & overhead

■ Group Chat Scalability

• Small groups (<100): fan-out on write — write one message per recipient to their mailbox

• Large groups (100-1000): fan-out on read — store single message, members pull; or Kafka topic per group

• Message ordering in groups: use logical clocks or Lamport timestamps; last-write-wins or version vectors

• WhatsApp approach: client-side ordering using server timestamps; no strict total order guarantees needed

■ End-to-End Encryption

• Signal Protocol: each device has identity key + signed prekey + one-time prekeys stored on server

• Sender encrypts with recipient's public key bundle; server stores only encrypted ciphertext — cannot read messages

• Key exchange on first message; subsequent messages use ephemeral Diffie-Hellman (forward secrecy)

• Group E2E: sender encrypts once with group key; group key distributed to members via individual E2E channels